TEAM 9 · 2026–2027

Adaptive Traffic Signal *Controller*

An IoT-Based Simulation for Traffic-Aware Signal Timing

Amrita Vishwa Vidyapeetham, Amritapuri Campus

B.Tech CSE · Semester VII · IoT Mini-Project

Samanthula Eswar
AM.SC.U4CSE23349

Rayudu Vamsi Krishna
AM.SC.U4CSE23344

Bantupalli Harsha Vardhan
AM.SC.U4CSE23210

Kota Karthik
AM.SC.U4CSE23229

Sammeta Pavan
AM.SC.U4CSE23350

A software-based traffic simulation

Fixed timing meets *changing* demand

- Fixed-time signals use preset timings.
- Traffic demand changes throughout the day.
- A fixed green may not match the current queue.
- Uneven demand can raise waiting time and queue length.

PROBLEM STATEMENT

How can a traffic signal use incoming traffic information to allocate green time more appropriately across four approaches – while monitoring command execution?

Six objectives, one *pipeline*

1

Simulate traffic readings for North, South, East and West.

2

Transfer readings and signal commands through MQTT.

3

Calculate green-time requests from traffic demand.

4

Store readings, decisions, events and command records.

5

Monitor traffic and controller behaviour on a dashboard.

6

Evaluate offline ML models and closed-loop safety behaviour.

Fixed-time vs *adaptive* simulation

FIXED-TIME APPROACH	PROPOSED ADAPTIVE SIMULATION
Preset green durations	Green-time request based on measured demand
Does not react directly to the current queue	Uses queue length, arrival rate and waiting time
Limited visibility into controller events	Dashboard and database record system activity
Timing not tied to actuator acknowledgement	Simulated commands checked using acknowledgements

Key idea: use current simulated traffic demand to request more suitable signal timings.

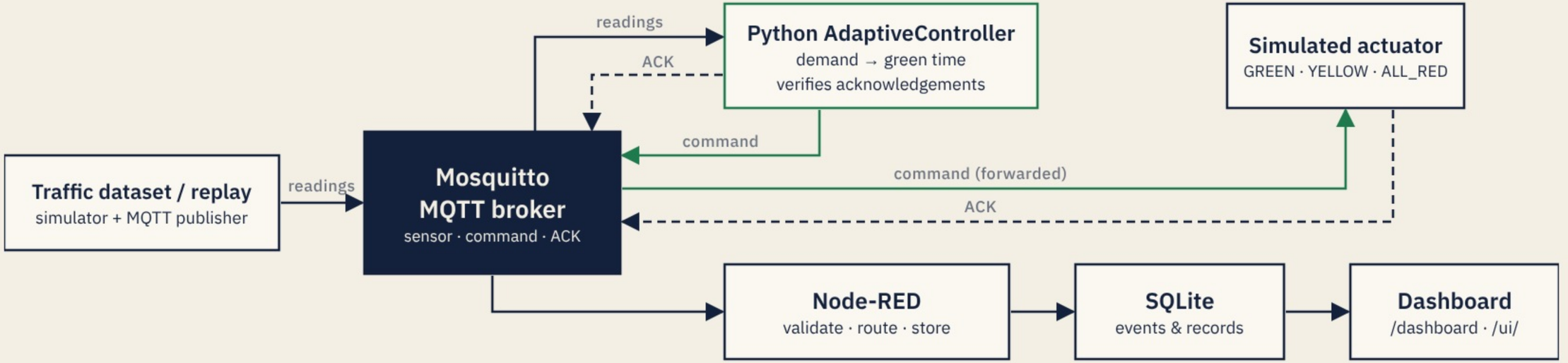
Team-prepared, *transparent* data

5,000 traffic readings	***1,250*** timestamps × 4 approaches	***≈1 min*** sampling interval	***4*** North · South · East · West

Reference traffic patterns → Team-prepared workbook → CSV files → Simulation & evaluation

Live system vs *offline* analysis

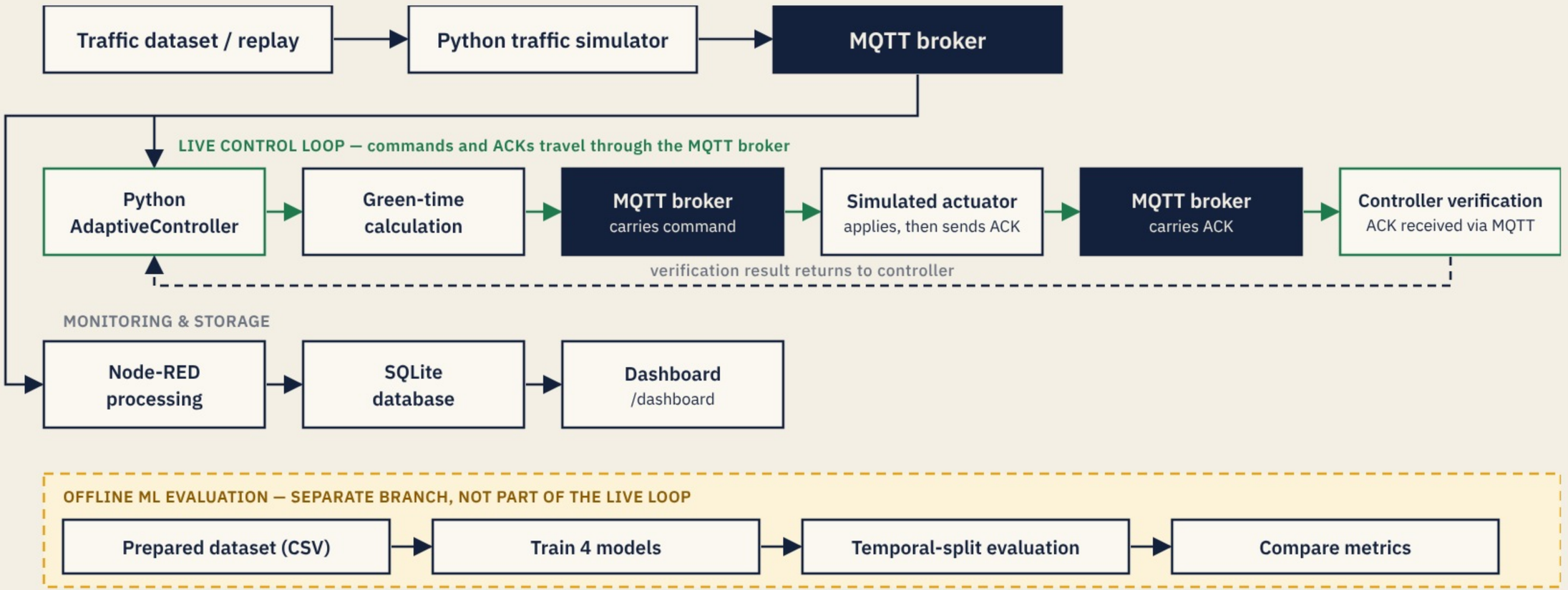
• LIVE SYSTEM — DETERMINISTIC PYTHON CONTROLLER



OFFLINE ANALYSIS — DOES NOT CONTROL LIVE SIGNAL TIMING

Prepared dataset (CSV) → Train 4 ML models → Temporal-split evaluation → Accuracy · macro F1 · HIGH recall

End-to-end *workflow*



Technology *stack*

LAYER	TECHNOLOGY / TOOL	ROLE
Programming	Python 3.13.7	Simulation, controller, analytics, tests
Messaging	MQTT · Mosquitto 2.1.2	Sensor and command communication
Flow integration	Node-RED 5.0.1	Message processing and dashboard flows
Runtime	Node.js 24.18.0	Node-RED runtime
Storage	SQLite	Local structured project records
Machine learning	scikit-learn 1.7.2	Offline model comparison
Numerical	NumPy 2.2.6	Data and numerical operations
Version control	Git and GitHub	Source-code management

MQTT, Node-RED and *SQLite*

MQTT COMMUNICATION

- Publishes sensor readings per approach.
- Carries signal command and acknowledgement messages.
- Separates producers and consumers through topics.

NODE-RED INTEGRATION

- Receives and validates incoming messages.
- Connects flows to storage and dashboard.
- Makes system activity visible during simulation.

SQLITE PERSISTENCE

- Sensor readings
- Signal decisions and signal events
- Actuator commands and system events

Adaptive signal *timing*

TRAFFIC DEMAND INPUTS

- Queue length
- Vehicle arrival rate
- Waiting time

DEMAND-BASED TIMING RULE

Requested green = Previous green × (Approach demand score ÷ Average demand score)

CONFIGURED TIMING LIMITS

10 s Minimum green	*60 s* Maximum green	*25 s* Default green
3 s Yellow	*2 s* All-red	

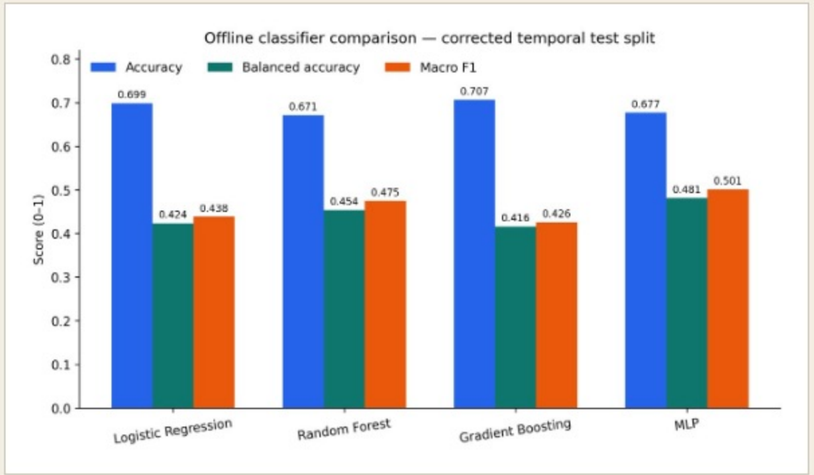
Four offline *models*

LOGISTIC REGRESSION	RANDOM FOREST	GRADIENT BOOSTING	MLP NEURAL NETWORK
StandardScaler; max_iter = 1,000	300 trees; max_features = sqrt; unweighted	150 estimators; learning rate 0.05; depth 2	Hidden layers (64, 32); alpha 0.0001; max_iter 500

Purpose: compare models for traffic-level classification and decision support. **Evaluated offline — they do not control the live signal cycle.**

Accuracy alone is *misleading*

MODEL	ACC.	BAL. ACC.	MACRO F1	HIGH REC.
Majority baseline	0.6780	—	—	—
Logistic Reg.	0.6987	0.4238	0.4384	0.1327
Random Forest	0.6713	0.4538	0.4747	0.2143
Gradient Boosting	**0.7067**	0.4163	0.4258	0.0714
MLP Neural Net	0.6773	**0.4815**	**0.5014**	**0.2857**



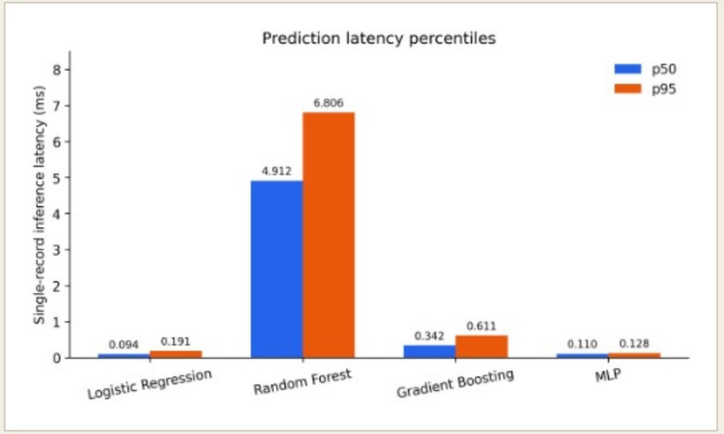
Temporal split: 3,492 train · 1,500 test · 4 boundary rows purged. Test support: LOW 1,017 · MEDIUM 385 · HIGH 98.

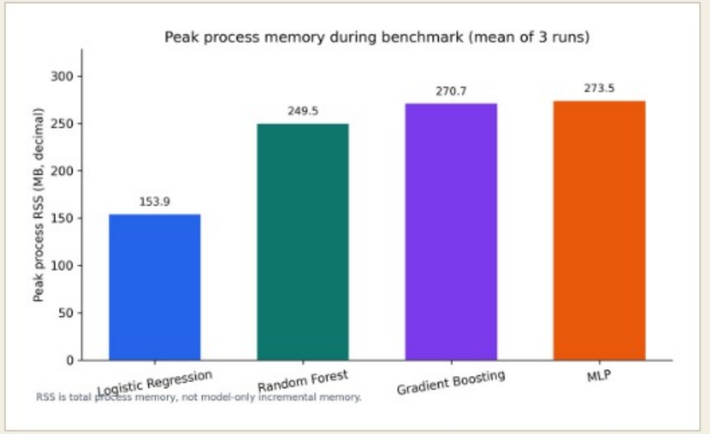
Gradient Boosting: best accuracy. MLP: best balanced accuracy, macro F1 and HIGH recall — yet its accuracy sits just below the majority baseline.

Computational *performance*

MODEL	FIT TIME (S)	P50 (MS)	P95 (MS)	ARTIFACT (MB)
Logistic Regression	0.0253	0.094	0.191	0.030
Random Forest	0.7969	4.912	6.806	53.972
Gradient Boosting	0.7152	0.342	0.611	0.361
MLP Neural Network	2.3105	0.110	0.128	0.129

Windows 11 · CPU-only · Python 3.13.7 · 12 logical CPUs · NumPy 2.2.6, scikit-learn 1.7.2. 3 training runs; single-row inference over 1,000 samples. Local benchmark, not end-to-end MQTT latency.





Closed-loop evaluation & *safety*

NORMAL OPERATION

37/37 phase acknowledgements accepted · zero violations on successful paths

STOP & RESET

4/4 unique RED commands acknowledged on each successful operation

FAULT INJECTION

Withheld ACKs → failed / unconfirmed. GREEN restart stays inhibited.

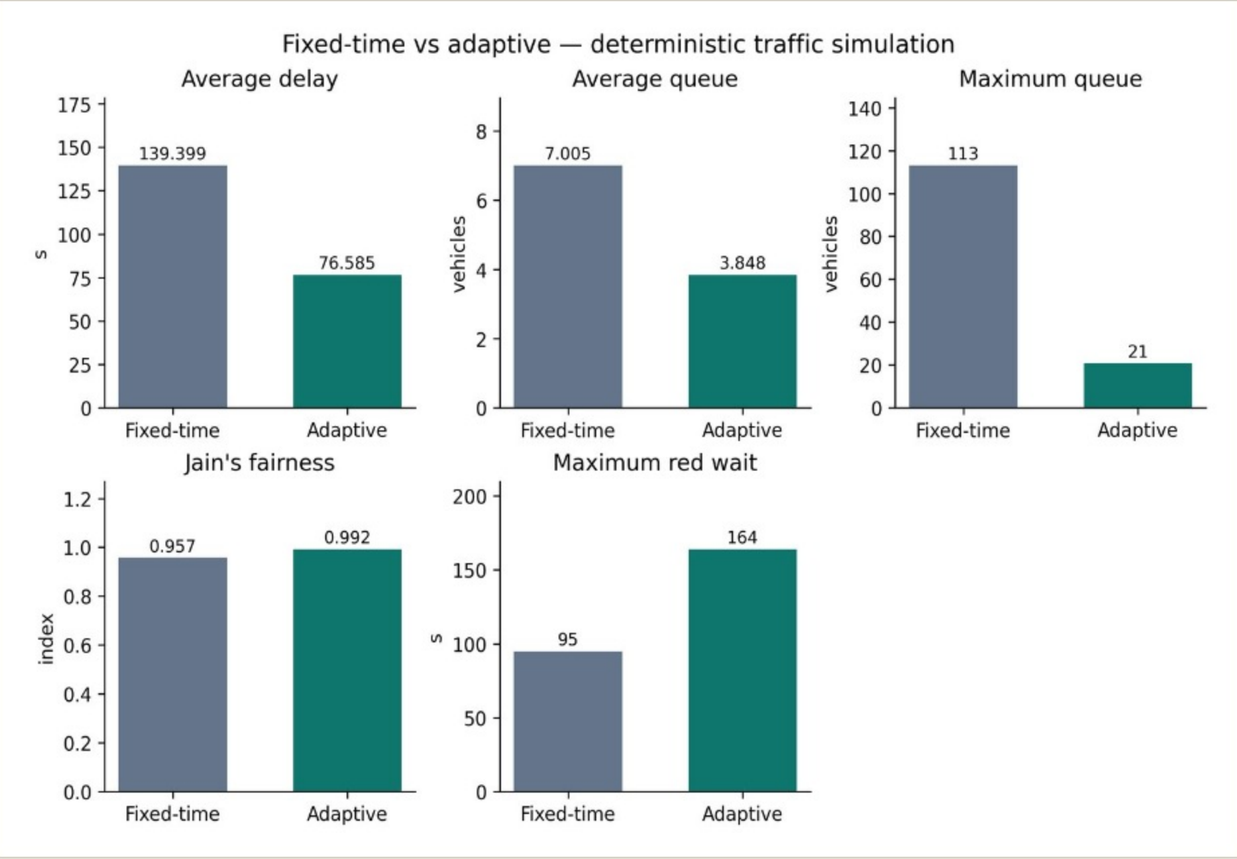


RUNNING → STOP / RESET → 4 × RED → 4 × ACK → CONFIRMED SAFE

Missing ACK → UNCONFIRMED · RESTART INHIBITED

200 automated tests passed, 0 failed. Software test results — not a real-world safety certification.

Fixed vs adaptive: a *trade-off*

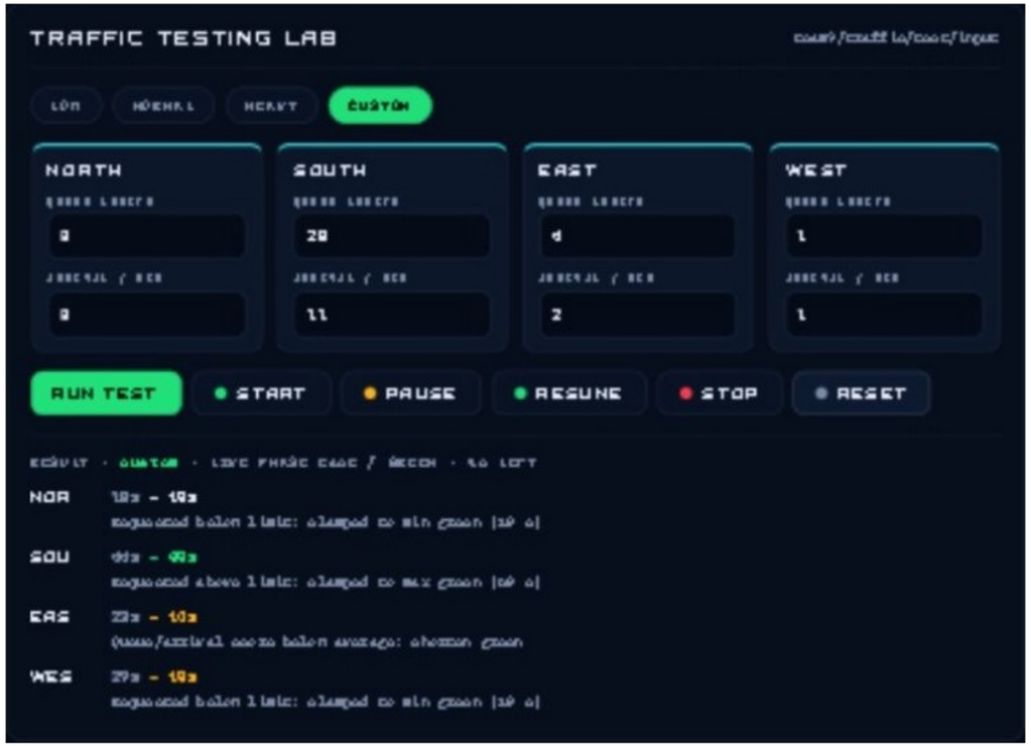


METRIC	FIXED	ADAPTIVE
Avg delay	139.399 s	76.585 s
Avg queue	7.005	3.848
Max queue	113	21
Jain fairness	0.9572	0.9923
Max red wait	95 s	**164 s**

Simulation results, not field data. Delay and queues fall, but maximum red wait rises 95 s → 164 s.

Live dashboard & *Testing Lab*





Conclusion, limitations & *next steps*

ACHIEVEMENTS

- End-to-end IoT-style traffic simulation
- Demand-based green-time calculation
- MQTT, Node-RED, SQLite and dashboard integrated
- Offline ML and closed-loop ACK evaluation
- Stop/Reset ACK handling and restart inhibition verified

LIMITATIONS

- Simulation only — no physical sensors or lights
- Team-prepared dataset; original source unavailable
- Simplified simulation, not field measurements
- ACK timing excludes real network delays
- No fuel or emission measurements

FUTURE WORK

- Real sensors and hardware-in-the-loop
- Recover dataset references
- Validate on real junction data
- Improve HIGH-traffic detection
- Test network delays and failures
- Study fairness and max red wait

References

Project source
github.com/eswarsamanthula/Adaptive_Traffic_Signal_Controller

Python
docs.python.org/3/

MQTT
mqtt.org

Eclipse Mosquitto
mosquitto.org/documentation/

Node-RED
nodered.org/docs/

Node.js
nodejs.org/docs/

SQLite
sqlite.org/docs.html

scikit-learn
scikit-learn.org/stable/

NumPy
numpy.org/doc/

Git and GitHub
git-scm.com/doc · docs.github.com

Thank *You*